Universidad Politécnica de Chiapas

# **SICTMRRMAGIEFH-EVA**: Sistema inteligente para calcular las tensiones mecánicas de un rostro robótico mediante algoritmos genéticos para la imitación de expresiones faciales humanas, basado en el caso EVA

## INTELIGENCIA ARTIFICIAL

Ingeniería en Software


Author:

Luis Alberto Batalla González

Docente

Dr. Carlos Alberto Díaz Hernández


Suchiapa, Chiapas, enero - abril de 2026

# Resumen

En este reporte se abordarán los fundamentos necesarios para la elaboración de un programa computacional que usa Algoritmos Genéticos para resolver una problemática especifica planteada en este mismo documento.

# Abstract

"This report will cover the fundamental concepts required to develop a computer program that utilizes Genetic Algorithms to solve a specific problem presented within this same document."

# Índice general

**1. SICTMRRMAGIEFH-EVA: Sistema inteligente para calcular las tensiones mecánicas de un rostro robótico mediante algoritmos genéticos para la imitación de expresiones faciales humanas, basado en el caso EVA 8**
1.1. Introducción . . . 8
1.2. Planteamiento del problema . . . 8
1.2.1. Contexto . . . 8
1.2.2. Problemática . . . 9
1.3. Propuesta de Solución . . . 10
1.3.1. Requerimientos y restricciones . . . 10
1.3.2. Análisis y Diseño . . . 10
1.3.3. Base de conocimientos . . . 15
1.3.4. Función de Aptitud . . . 15
1.4. Implementación . . . 17
1.4.1. Función de Aptitud . . . 17
1.4.2. Inicialización . . . 17
1.4.3. Generación de parejas . . . 18
1.4.4. Cruza . . . 18
1.4.5. Mutación . . . 19
1.4.6. Poda . . . 19
1.5. Resultados . . . 20
1.5.1. Evolución del Error y Diversidad . . . 20
1.5.2. Evolución de la Aptitud (Fitness) . . . 21
1.5.3. Validación Cruzada y Generalización . . . 22
1.5.4. Mejor Individuo y Resultados Contundentes . . . 22

**2. Conclusiones y recomendaciones 24**
2.1. Conclusiones . . . 24
2.1.1. Aprendizajes . . . 24
2.1.2. Mejoras en Retrospectiva . . . 24
2.1.3. Nuevos Proyectos . . . 24
2.2. Recomendaciones . . . 24

**3. Anexos 25**
3.1. Base de Conocimientos y Modelo del Robot . . . 25
3.2. Interfaz del Sistema en Ejecución . . . 25

3.3. Repositorio de Código Fuente . . . . . 25

# Índice de figuras

1.1. Imagen representativa extraída del artículo en el que se basó el programa [1]. . . . . 9
1.2. Imagen representativa extraída del artículo en el que se basó el programa [1]. . . . . 9
1.3. Estructura del cromosoma binario del individuo. Cada segmento de 10 bits codifica la tensión normalizada de un actuador facial. . . . . 12
1.4. Curvas de decaimiento del error y mantenimiento de la diversidad (Sesiones 1 y 2). . . . . 20
1.5. Curvas de decaimiento del error y mantenimiento de la diversidad (Sesiones 3 y 4). . . . . 21
1.6. Trayectorias de aptitud del mejor, peor y promedio poblacional. . . . . 22
1.7. Resultados del análisis K-Fold para validación del algoritmo. . . . . 22
1.8. Podio de resultados: mejores vectores de tensión $[m_1, \ldots, m_6]$ encontrados. 23

3.1. Captura del sistema EVA operando en tiempo real. . . . . 25

# Índice de bloques de código

1.1. Segmento de código que implementa la función de aptitud (Eq. 1.5). . . 17
1.2. Segmento de código que implementa la inicialización. . . . . 17
1.3. Segmento de código que implementa la estrategia de generación de parejas. 18
1.4. Segmento de código que implementa la cruza de las parejas. . . . . 18
1.5. Segmento de código que implementa la mutación de la descendencia. . . 19
1.6. Segmento de código que implementa de la poda de la población. . . . . 19

# Índice de tablas

1.1. Requerimientos y restricciones del sistema EVA. . . . . 10
1.2. Variables de decisión . . . . . 11
1.3. Variables a optimizar . . . . . 13
1.4. Variables y objetivos de optimización. . . . . 14

# Capítulo 1

# SICTMRRMAGIEFH-EVA: Sistema inteligente para calcular las tensiones mecánicas de un rostro robótico mediante algoritmos genéticos para la imitación de expresiones faciales humanas, basado en el caso EVA

## 1.1. Introducción

El proyecto fue desarrollado como requisito para evidenciar las competencias adquiridas sobre los Algoritmos Genéticos en el marco de la asignatura Inteligencia Artificial de Ingeniería en Software de la Universidad Politécnica de Chiapas.

Resolviendo una problemática real a través del uso del enfoque de algoritmos evolutivos, para resolver problemas no triviales de optimización.

## 1.2. Planteamiento del problema

### 1.2.1. Contexto

Actualmente, los robots trabajan cada vez más cerca de las personas en lugares como asilos, hospitales, fábricas y hogares. Sin embargo, casi todos estos robots tienen rostros estáticos hechos de plástico o metal rígido, incapaces de mostrar emociones.

Esto genera una importante barrera de confianza humano-robot: a las personas les resulta difícil conectar o sentirse cómodas colaborando con máquinas que tienen rostros inexpresivos (estilo "jugador de póquer"). Para solucionar este problema acústico y social, proyectos como el robot EVA proponen crear rostros robóticos de silicona suave, controlados internamente por decenas de pequeños motores y cables que actúan como "músculos artificiales".

El gran problema técnico surge al intentar controlar esta nueva tecnología física. Coordinar mecánicamente la tensión exacta de más de 40 pequeños motores para recrear una sonrisa o un gesto humano natural es tan complejo que resulta prácticamente imposible de programar a mano usando reglas matemáticas tradicionales. La elasticidad de la silicona, el desgaste de las piezas y la fricción hacen que los movimientos del rostro sean altamente impredecibles.1.2.

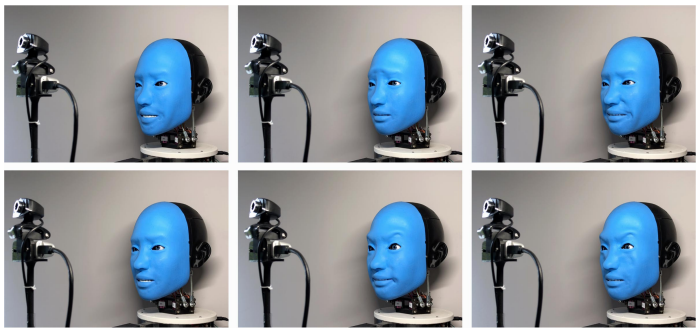


Figura 1.1: Imagen representativa extraída del artículo en el que se basó el programa [1].

### 1.2.2. Problemática

La incapacidad de calcular y coordinar con precisión las tensiones mecánicas de los múltiples músculos artificiales provoca que el rostro de silicona del robot genere gesticulaciones espasmódicas, erráticas o gravemente antinaturales. Este fallo motriz desencadena un fuerte rechazo psicológico en los usuarios humanos (fenómeno conocido como el "valle inquietante"), lo que destruye cualquier posibilidad de empatía o confianza humano-máquina y, en última instancia, provoca el fracaso comercial y operativo del robot en entornos donde su labor asistencial, médica o colaborativa es crítica.

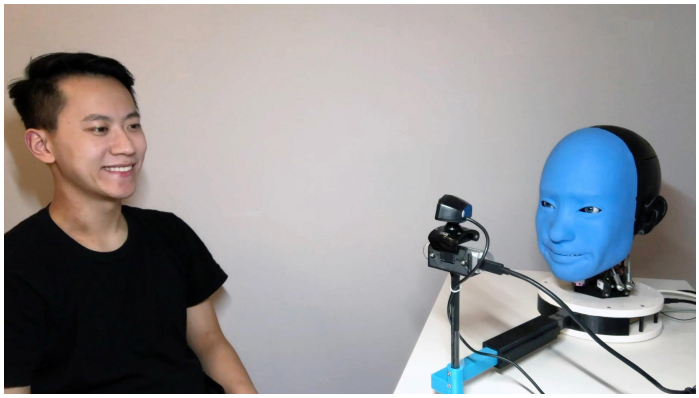


Figura 1.2: Imagen representativa extraída del artículo en el que se basó el programa [1].

## 1.3. Propuesta de Solución

El proyecto consiste en el desarrollo de un sistema de simulación basado en algoritmos genéticos que toma como entrada las coordenadas faciales en tiempo real de un usuario capturado por video, procesándolas para calcular la tensión motriz óptima de los múltiples cables y actuadores de un rostro robótico blando (basado en la arquitectura EVA), buscando replicar fielmente la expresión humana en un gemelo digital 2D para superar las limitaciones de la cinemática inversa tradicional.

### 1.3.1. Requerimientos y restricciones

La operatividad del sistema de imitación gestual del robot EVA impone un conjunto de requerimientos funcionales y restricciones paramétricas que delimitan el espacio de soluciones válidas. Dado que el Algoritmo Genético debe converger dentro de la latencia de un solo fotograma de video para garantizar fluidez visual, el presupuesto temporal constituye la restricción primaria del sistema. Adicionalmente, los actuadores mecánicos de silicona poseen límites físicos de operación normalizados: toda tensión calculada debe estar estrictamente acotada al intervalo continuo $[0.0, 1.0]$ para evitar daños estructurales en el elastómero. La codificación binaria, por su parte, impone una resolución discreta que acota la granularidad del control motriz. En la Tabla 1.1 se sintetizan las restricciones del problema con sus respectivos valores en notación implícita o explícita.

| **Restricción** | **Descripción** | **Valor / Notación** |
|---|---|---|
| Tiempo real | Presupuesto máximo de cómputo del AG por cada fotograma capturado | $t_{AG} \leq 33.3$ ms |
| Rango de tensión | Dominio válido de cada variable de decisión $m_i$ | $m_i \in [0.0, 1.0],\ \forall i \in \{1, \ldots, 6\}$ |
| Resolución mínima | Sensibilidad de control deseada sobre cada actuador | $\delta = \{x \in R^+ \mid x \geq 10^{-3}\}$ |
| Espacio de búsqueda | Cardinalidad total del espacio genotípico | $\lvert\mathcal{S}\rvert = 2^L = 2^{60}$ combinaciones |
| Resolución de captura | Dimensiones de los fotogramas de entrada | $320 \times 240$ px |
| Convergencia | Umbral de error cuadrático medio aceptable | $E < \epsilon = 0.001$ |
| Diversidad poblacional | Rango saludable de diversidad normalizada de Hamming | $D_{norm} \in [0.2, 0.4]$ |
| Detección facial | Nivel mínimo de confianza de MediaPipe para aceptar un fotograma | Confianza $\geq 0.5$ |

Tabla 1.1: Requerimientos y restricciones del sistema EVA.

### 1.3.2. Análisis y Diseño

#### Variables de Decisión

Se identificaron $n = 6$ variables de decisión que representan la tensión mecánica normalizada de cada actuador del rostro robótico. Cada variable codifica la fracción de contracción de un músculo sintético específico, según se describe en la Tabla 1.2.

| Id. | Variable de Decisión | Descripción | Efecto |
|---|---|---|---|
| $m_1$ | Tensión mandibular (apertura de boca) | Controla el desplazamiento vertical de la mandíbula inferior del rostro robótico | Cuando $m_1 \to 1.0$, la boca se abre completamente; cuando $m_1 \to 0.0$, permanece cerrada. |
| $m_2$ | Tensión cigomática izquierda (comisura labial izq.) | Controla la elevación o descenso del extremo izquierdo de la boca, determinando la curvatura de la sonrisa en ese lado | Valores altos generan una sonrisa asimétrica izquierda; valores bajos mantienen la comisura neutra o descendente. |
| $m_3$ | Tensión cigomática derecha (comisura labial der.) | Controla la elevación o descenso del extremo derecho de la boca | Equivalente simétrico de $m_2$; la combinación $m_2 \approx m_3 \approx 1.0$ produce una sonrisa bilateral completa. |
| $m_4$ | Tensión orbicular izquierda (párpado izq.) | Controla el grado de apertura o cierre del ojo izquierdo | Cuando $m_4 \to 0.0$, el ojo se cierra (pestañeo); cuando $m_4 \to 1.0$, el ojo está completamente abierto (sorpresa). |
| $m_5$ | Tensión orbicular derecha (párpado der.) | Controla el grado de apertura o cierre del ojo derecho | Equivalente simétrico de $m_4$; la combinación $m_4 \approx m_5 \approx 0.0$ reproduce un doble pestañeo simultáneo. |
| $m_6$ | Tensión frontal (cejas) | Controla la elevación o el fruncimiento de las cejas | Cuando $m_6 \to 1.0$, las cejas se elevan expresando sorpresa; cuando $m_6 \to 0.0$, se fruncen expresando enojo o concentración. |

Tabla 1.2: Variables de decisión

Todas las variables de decisión comparten el dominio continuo $m_i \in [0.0, 1.0]$, donde $0.0$ representa elongación total (músculo completamente relajado) y $1.0$ representa contracción máxima (músculo completamente contraído). Este rango normalizado permite que el sistema sea independiente de las unidades físicas del actuador específico.

### Solución o Individuo

Cada individuo del Algoritmo Genético codifica una configuración completa del rostro robótico como un **cromosoma binario de $L = 60$ bits**. La estructura de datos empleada es un vector lineal de bits, donde cada segmento consecutivo de $k = 10$ bits representa la tensión de un actuador específico. Esta codificación permite $2^{10} = 1024$ niveles discretos

por actuador, logrando una resolución real de $\delta_{real} = \frac{1.0}{1023} \approx 0.000978$.

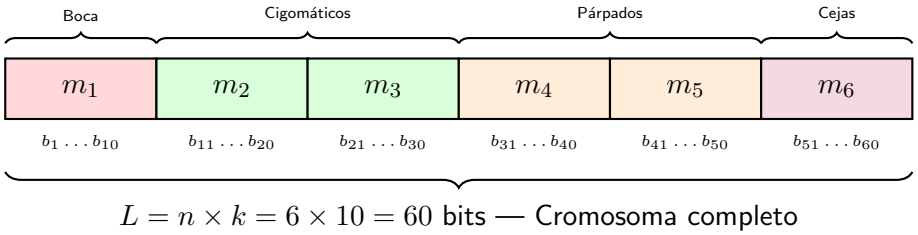



Figura 1.3: Estructura del cromosoma binario del individuo. Cada segmento de 10 bits codifica la tensión normalizada de un actuador facial.

La **decodificación** del segmento de bits del actuador $i$ al valor real $m_i$ se realiza en dos pasos:

1. Conversión del segmento binario a valor decimal: $d_i = \sum_{j=0}^{k-1} b_j \cdot 2^{k-1-j}$
2. Mapeo al rango real: $m_i = a + d_i \cdot \frac{b-a}{2^k-1} = 0.0 + d_i \cdot \frac{1.0}{1023}$

**Restricciones sobre el individuo:**

- Cada gen decodificado debe satisfacer $m_i \in [0.0, 1.0],\ \forall i \in \{1, \dots, 6\}$. La codificación binaria garantiza esto por construcción.
- No existen restricciones de dependencia entre actuadores: cada $m_i$ es independiente, lo que significa que toda combinación de tensiones constituye una configuración válida del sistema.
- El tamaño del cromosoma es fijo ($L = n \times k = 60$ bits) y no varía durante la ejecución.

### Variables a optimizar

Con base en las variables de decisión mencionadas en el apartado 1.3.2, se definieron $m = 5$ variables a optimizar que cuantifican el rendimiento del sistema, según se describen en la Tabla 1.3.

| Id. | Variable a Optimizar | Descripción | Variables de decisión involucradas |
|---|---|---|---|
| $E$ | Error de imitación facial (MSE) | Se determina calculando la distancia cuadrática media entre el vector objetivo $T$ (expresión del usuario humano) y el vector de tensiones $M$ del mejor individuo | Para su cálculo se utilizan todas las variables $m_1, \ldots, m_6$ comparadas con sus correspondientes valores objetivo $t_1, \ldots, t_6$ extraídos por MediaPipe. |
| $F$ | Aptitud de convergencia (Fitness) | Transformación inversa del error que el AG maximiza para guiar la selección evolutiva | Se calcula directamente a partir de $E$, que a su vez depende de $m_1, \ldots, m_6$. |
| $V$ | Velocidad de convergencia generacional | Número de generaciones requeridas por el AG para alcanzar un umbral de error aceptable $E < \epsilon$ ante una expresión facial nueva | Depende indirectamente de la distribución inicial de $m_1, \ldots, m_6$ en la población y de los hiperparámetros $p_c$, $p_m$. |
| $S$ | Estabilidad inter-fotograma | Mide la variación promedio del vector $[m_1, \ldots, m_6]$ del mejor individuo entre fotogramas consecutivos de video | Compara los valores de $m_1, \ldots, m_6$ del mejor individuo entre los fotogramas $f$ y $f-1$. |
| $D$ | Diversidad poblacional | Dispersión genética de la población medida por la distancia de Hamming promedio entre todos los pares de cromosomas | Evalúa la representación binaria de todas las variables $m_1, \ldots, m_6$ de todos los individuos de la generación actual. |

Tabla 1.3: Variables a optimizar

El **error de imitación facial** ($E$) se calcula a través de la Ecuación 1.1.

$$E(M,T) = \frac{1}{n}\sum_{i=1}^{n}(t_i - m_i)^2 \tag{1.1}$$

Donde $n = 6$ es el número de actuadores, $t_i$ es el valor objetivo del actuador $i$ extraído por MediaPipe y $m_i$ es la tensión decodificada del cromosoma del individuo. Un valor de $E = 0$ indica imitación perfecta; $E_{max} = 1.0$ cuando todos los actuadores están en el extremo opuesto al objetivo.

Se busca que el $E$ sea **minimizado**.

La **aptitud de convergencia** ($F$) se calcula a través de la Ecuación 1.2.

$$F(M,T) = \frac{1}{1 + E(M,T)} \tag{1.2}$$

Donde $E$ es el error de imitación. Cuando $E \to 0$, entonces $F \to 1$ (aptitud máxima). Cuando $E$ es grande, $F \to 0$.

Se busca que $F$ sea **maximizado**.

La **estabilidad inter-fotograma** ($S$) se calcula a través de la Ecuación 1.3.

$$S^{(f)} = \frac{1}{n} \sum_{i=1}^{n} \left( m_i^{(f)} - m_i^{(f-1)} \right)^2 \tag{1.3}$$

Donde $m_i^{(f)}$ es la tensión del actuador $i$ del mejor individuo en el fotograma $f$ y $m_i^{(f-1)}$ es la tensión del mismo actuador en el fotograma anterior. Valores de $S > 0.2$ indican movimientos espasmódicos indeseados en el avatar robótico.

Se busca que $S$ sea **minimizado**.

La **diversidad poblacional** ($D$) se calcula a través de la Ecuación 1.4.

$$D^{(g)} = \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} d_H(I_i, I_j) \tag{1.4}$$

Donde $d_H(I_i, I_j)$ es la distancia de Hamming entre los cromosomas de los individuos $I_i$ e $I_j$ (número de bits en los que difieren). La diversidad normalizada se obtiene como $D_{norm} = D/L$.

Se busca que $D_{norm}$ sea **equilibrado** dentro del rango $[0.2, 0.4]$: diversidad demasiado baja provoca convergencia prematura (mínimos locales), diversidad demasiado alta impide convergencia (expresión inestable).

### Objetivos de Optimización

Como resumen de las variables de optimización, mencionadas en el apartado 1.3.2, se presenta la Tabla 1.4.

| **Identificador** | **Fórmula** | **Objetivo de Optimización** | **Valores** |
|---|---|---|---|
| $E$ | Ec. 1.1 | Minimizar | $E \in [0,1]$; $E = 0$ imitación perfecta |
| $F$ | Ec. 1.2 | Maximizar | $F \in (0,1]$; $F = 1$ aptitud máxima |
| $V$ | Generaciones hasta $E < \epsilon$ | Minimizar | $V \in \{1, 2, \ldots, G_{max}\}$ |
| $S$ | Ec. 1.3 | Minimizar | $S \geq 0$; $S < 0.05$ transición suave |
| $D$ | Ec. 1.4 | Equilibrar | $D_{norm} \in [0.2, 0.4]$ rango saludable |

Tabla 1.4: Variables y objetivos de optimización.

### 1.3.3. Base de conocimientos

El sistema diseñado usa una base de conocimiento implementada como un conjunto de tablas estáticas dentro de un archivo SQLite (`eva_conocimiento.db`) que se utiliza para definir el modelo del robot, aplicar las reglas de transformación cinemática y para encontrar el valor de las variables de optimización. Estas tablas son pobladas una única vez antes de iniciar la ejecución y permanecen en modo solo-lectura durante toda la sesión del AG, garantizando que el algoritmo nunca altere las reglas del problema que intenta resolver.

- **Modelo del robot** (`robot_modelo`): Es una tabla de datos donde cada fila define un actuador del rostro robótico. Las columnas enlistan los atributos: identificador del actuador ($m_1, \ldots, m_6$), nombre biomecánico, rol funcional, número de bits por gen ($k = 10$) y los límites de tensión $[0.0, 1.0]$. Determina la dimensión del cromosoma utilizado por el AG.

- **Mapeo cinemático** (`mapeo_cinematico`): Conjunto de reglas de transformación que describen cómo cada combinación de tensiones de los actuadores desplaza visualmente los puntos clave del rostro en el gemelo digital (Pygame). Por ejemplo: si $m_2 = 1.0$ y $m_3 = 1.0$, ambas comisuras labiales ascienden y el sistema renderiza una sonrisa completa.

- **Expresiones de referencia Ekman** (`expresiones_ekman`): Catálogo de 6 vectores de tensión ideales $[m_1, \ldots, m_6]$ correspondientes a las emociones básicas universales (alegría, tristeza, sorpresa, miedo, enojo y asco), según la taxonomía de Paul Ekman. Se utilizan como referencia de calibración y para validación cruzada.

- **Parámetros biomecánicos del material** (`parametros_material`): Constantes que modelan las propiedades físicas del elastómero simulado, como el coeficiente de elasticidad y los límites de deformación, que escalan la señal de los actuadores hacia el movimiento de píxeles en pantalla.

- **Configuración del AG** (`config_ag`): Valores por defecto del sistema genético, como tamaño de población ($N$), probabilidad de cruza ($p_c$), probabilidad de mutación ($p_m$), máximo de generaciones ($G_{max}$), umbral de error ($\epsilon$), tamaño del torneo ($k_t$) y número de élites ($e$). Definidos por el usuario antes de la ejecución como punto de partida controlable del experimento.

Esta separación arquitectural garantiza que modificar el modelo del robot —por ejemplo, escalar de 6 a 12 actuadores— requiera únicamente alterar filas en la tabla `robot_modelo`, sin modificar una sola línea del algoritmo genético.

### 1.3.4. Función de Aptitud

Tomando como referencia la Tabla 1.4, las soluciones generadas por el Algoritmo Genético se evalúan con el objetivo de identificar cuál es mejor en términos de fidelidad gestual, con este fin se diseñó la función de aptitud (Ecuación 1.5).

$$Aptitud(Individuo) = \frac{1}{1 + E(M_{Individuo},\ T)} \tag{1.5}$$

Dado que el $E$ (Error de imitación) es una variable a minimizar, se debe tomar en cuenta que entre más grande sea, la solución codificada en el individuo es menos buena por lo que se propone que pase dividiendo, pero dado que el valor mínimo del error es $E = 0$ (imitación perfecta), se le agrega un $1$ para evitar divisiones por $0$. De este modo:

- Cuando $E = 0$ (imitación perfecta): $Aptitud = \frac{1}{1+0} = 1.0$ (aptitud máxima).
- Cuando $E \to \infty$ (error enorme): $Aptitud \to 0$ (aptitud nula).
- La función es monótonamente decreciente respecto a $E$: a menor error, mayor aptitud.

Esta transformación hiperbólica garantiza que $Aptitud \in (0, 1]$ para todo individuo válido, proporcionando al mecanismo de selección por torneo un criterio de comparación estrictamente positivo y acotado superiormente.

## 1.4. Implementación

### 1.4.1. Función de Aptitud

En el listado 1.1 se muestra la implementación de la función de aptitud, si se invocan métodos para calcular las variables objetivos deben agregarse como listados separados cada uno y ser referenciados en el texto.

```
def calcular_aptitud(mse: float) -> float:
    """
     4 .2     Transforma el error E en aptitud F mediante inversión hiperbólica.

    F(M, T) = 1 / (1 + E(M, T))

    Propiedades:
        - F     (0, 1].
        - F = 1 cuando E = 0 (imitación perfecta     aptitud máxima).
        - F     0 cuando E        .
        - Monótonamente decreciente respecto a E.

    Args:
        mse: Error Cuadrático Medio (E     0).

    Returns:
        Valor de fitness F     (0, 1].
    """
    return 1.0 / (1.0 + mse)
```

Listing 1.1: Segmento de código que implementa la función de aptitud (Eq. 1.5).

### 1.4.2. Inicialización

Tomando en cuenta la naturaleza de la problemática que está resolviendo, explique qué procesos realiza en la inicialización.

```
@dataclass
class Individuo:
    cromosoma: str = ""
    tensiones: List[float] = field(default_factory=list)
    error: float = float('inf')
    aptitud: float = 0.0

    def __str__(self) -> str:
        tens_str = ", ".join(f"{t:.4f}" for t in self.tensiones)
        return (f"Cromosoma: {self.cromosoma[:20]}... | "
                f"M=[{tens_str}] | E={self.error:.6f} | F={self.aptitud:.6f}")


def generar_cromosoma_aleatorio(longitud: int) -> str:
    return ''.join(random.choice('01') for _ in range(longitud))
```

```python


def generar_poblacion(N: int, params: dict) -> List[Individuo]:
    L = params['longitud_cromosoma']
    poblacion = []
    for _ in range(N):
        cromosoma = generar_cromosoma_aleatorio(L)
        individuo = Individuo(cromosoma=cromosoma)
        poblacion.append(individuo)
    return poblacion
```

Listing 1.2: Segmento de código que implementa la inicialización.

### 1.4.3. Generación de parejas

Explique y justifique la estrategia de generación de parejas elegida. Es imprescindible que se apoye de un diagrama para explicar la estrategia.

```python
def seleccion_torneo(poblacion: List[Individuo], k: int = 3) -> Individuo:
    competidores = random.sample(poblacion, min(k, len(poblacion)))
    return max(competidores, key=lambda ind: ind.aptitud)


def emparejar(poblacion: List[Individuo], k: int = 3) -> List[Tuple[Individuo, Individuo]]:
    num_parejas = len(poblacion) // 2
    parejas = []
    for _ in range(num_parejas):
        padre1 = seleccion_torneo(poblacion, k)
        padre2 = seleccion_torneo(poblacion, k)
        parejas.append((padre1, padre2))
    return parejas
```

Listing 1.3: Segmento de código que implementa la estrategia de generación de parejas.

### 1.4.4. Cruza

Explique y justifique la estrategia de cruza de las parejas. Es imprescindible que se apoye de un diagrama para explicar la estrategia.

```python
def cruza_un_punto(padre1: Individuo, padre2: Individuo,
                   pc: float = 0.8) -> Tuple[Individuo, Individuo]:
    if random.random() >= pc:
        return Individuo(cromosoma=padre1.cromosoma), Individuo(cromosoma=padre2.cromosoma)
    L = len(padre1.cromosoma)
    c = random.randint(1, L - 1)
    hijo1 = Individuo(cromosoma=padre1.cromosoma[:c] + padre2.cromosoma[c:])
    hijo2 = Individuo(cromosoma=padre2.cromosoma[:c] + padre1.cromosoma[c:])
```

```
    return hijo1, hijo2
```

Listing 1.4: Segmento de código que implementa la cruza de las parejas.

### 1.4.5. Mutación

Explique y justifique la estrategia de mutación de la descendencia. Es imprescindible que se apoye de un diagrama para explicar la estrategia.

```
def mutar(individuo: Individuo, pm: float = 0.01) -> Individuo:
    bits = list(individuo.cromosoma)
    for i in range(len(bits)):
        if random.random() < pm:
            bits[i] = '1' if bits[i] == '0' else '0'
    return Individuo(cromosoma=''.join(bits))
```

Listing 1.5: Segmento de código que implementa la mutación de la descendencia.

### 1.4.6. Poda

Explique y justifique la estrategia la poda de la población. Es imprescindible que se apoye de un diagrama para explicar la estrategia.

```
def podar(poblacion: List[Individuo], N: int, f_min: float = 0.0,
    params: dict = None) -> List[Individuo]:
    if f_min > 0.0:
        poblacion = [ind for ind in poblacion if ind.aptitud >=
    f_min]
    poblacion_ordenada = sorted(poblacion, key=lambda ind: ind.
    aptitud, reverse=True)
    if len(poblacion_ordenada) >= N:
        return poblacion_ordenada[:N]
    resultado = list(poblacion_ordenada)
    faltantes = N - len(resultado)
    if params is not None:
        L = params['longitud_cromosoma']
        for _ in range(faltantes):
            resultado.append(Individuo(cromosoma=
    generar_cromosoma_aleatorio(L)))
    return resultado
```

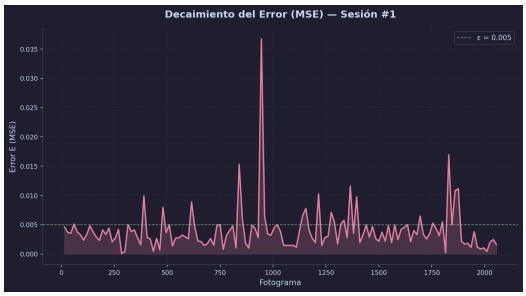
Listing 1.6: Segmento de código que implementa de la poda de la población.

## 1.5. Resultados

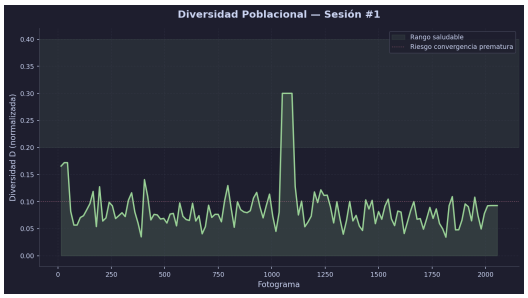
A continuación, se presentan los resultados obtenidos del sistema *SICTMRRMAGIEFH-EVA*. Con el fin de evaluar el rendimiento del Algoritmo Genético ante la naturaleza no lineal de la silicona y el dinamismo de las expresiones humanas, se realizaron cuatro sesiones experimentales con distintos perfiles de gesticulación.

### 1.5.1. Evolución del Error y Diversidad

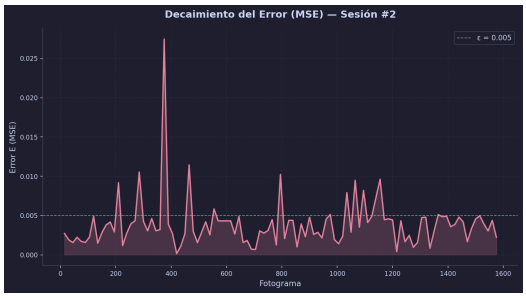
Para cada sesión, se monitoreó el Error Cuadrático Medio ($E$) y la Diversidad Poblacional ($D$). El objetivo es minimizar el error manteniendo una diversidad que evite la convergencia prematura.



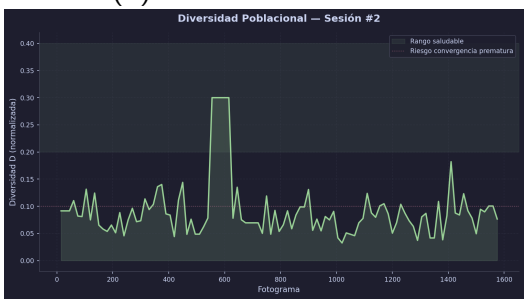

(a) Error Sesión 1




(b) Diversidad Sesión 1

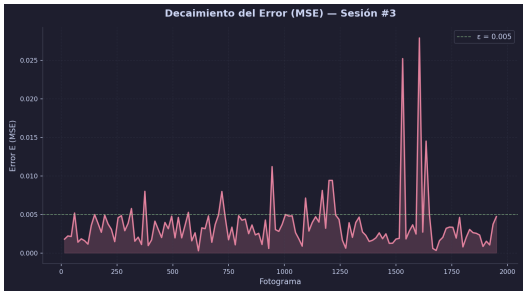



(c) Error Sesión 2

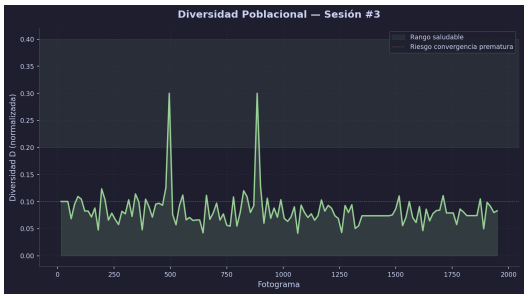



(d) Diversidad Sesión 2

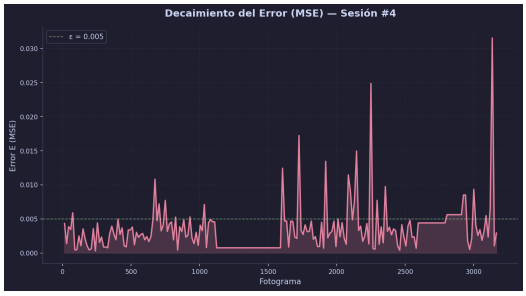
Figura 1.4: Curvas de decaimiento del error y mantenimiento de la diversidad (Sesiones 1 y 2).



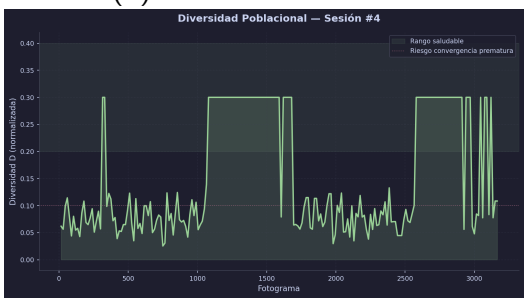

(a) Error Sesión 3




(b) Diversidad Sesión 3




(c) Error Sesión 4

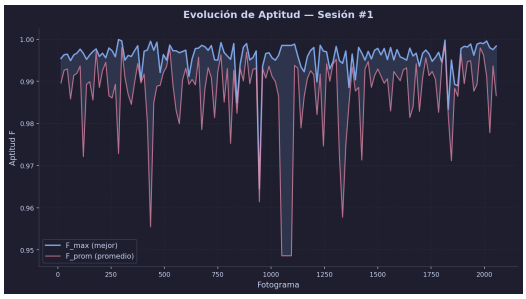



(d) Diversidad Sesión 4

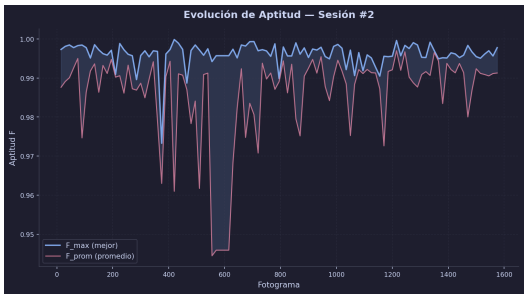
Figura 1.5: Curvas de decaimiento del error y mantenimiento de la diversidad (Sesiones 3 y 4).

### 1.5.2. Evolución de la Aptitud (Fitness)

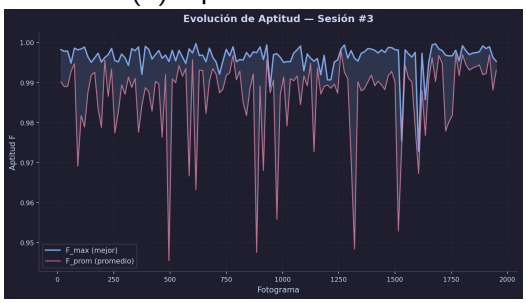
Se presenta la aptitud del mejor y peor individuo, así como el promedio poblacional antes de la poda. Se observa una presión selectiva efectiva que eleva el promedio generacional en todos los escenarios.



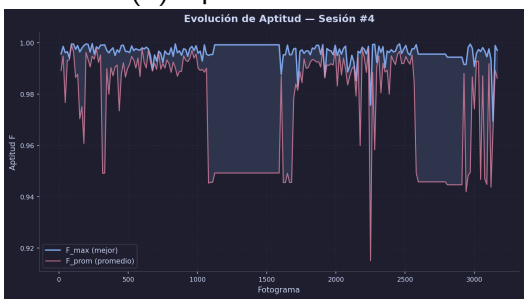

(a) Aptitud Sesión 1




(b) Aptitud Sesión 2




(c) Aptitud Sesión 3

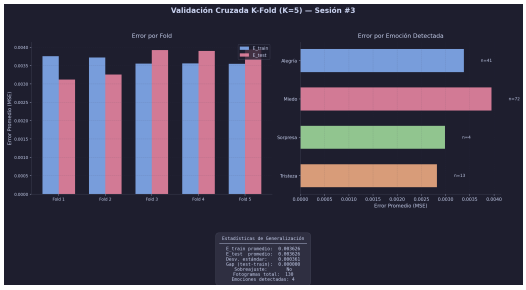



(d) Aptitud Sesión 4

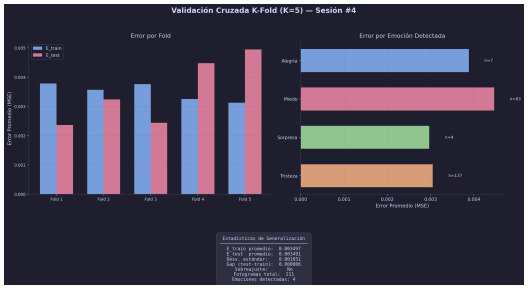
Figura 1.6: Trayectorias de aptitud del mejor, peor y promedio poblacional.

### 1.5.3. Validación Cruzada y Generalización

En las sesiones 3 y 4 se aplicó un análisis de validación cruzada ($K$-Fold) para evaluar la robustez de los parámetros ante cambios en la secuencia de entrada del usuario. Los resultados demuestran un error de generalización consistente con los objetivos de diseño.




(a) Sesión 3

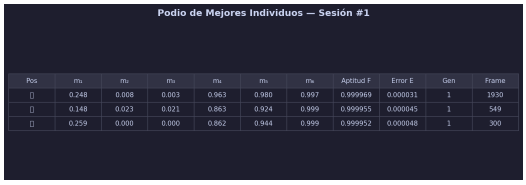



(b) Sesión 4

Figura 1.7: Resultados del análisis K-Fold para validación del algoritmo.

### 1.5.4. Mejor Individuo y Resultados Contundentes

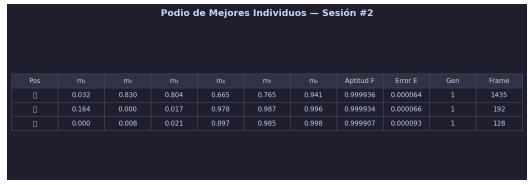
Finalmente, se visualiza el "Podio" de cada sesión, mostrando los tres mejores individuos encontrados. Estas soluciones representan las configuraciones de tensión del rostro robótico que lograron la imitación más fiel.



Podio de Mejores Individuos — Sesión #1

| Pos | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | Aptitud F | Error E | Gen | Frame |
|---|---|---|---|---|---|---|---|---|---|---|
| [illegible] | 0.248 | 0.008 | 0.003 | 0.963 | 0.980 | 0.997 | 0.999969 | 0.000031 | 1 | 1930 |
| [illegible] | 0.148 | 0.023 | 0.021 | 0.863 | 0.924 | 0.999 | 0.999955 | 0.000045 | 1 | 549 |
| [illegible] | 0.259 | 0.000 | 0.000 | 0.862 | 0.944 | 0.999 | 0.999952 | 0.000048 | 1 | 300 |

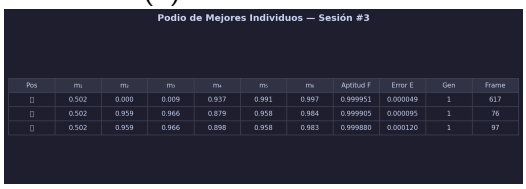
(a) Podio Sesión 1



Podio de Mejores Individuos — Sesión #2

| Pos | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | Aptitud F | Error E | Gen | Frame |
|---|---|---|---|---|---|---|---|---|---|---|
| [illegible] | 0.032 | 0.830 | 0.804 | 0.665 | 0.765 | 0.941 | 0.999936 | 0.000064 | 1 | 1435 |
| [illegible] | 0.164 | 0.000 | 0.017 | 0.978 | 0.987 | 0.996 | 0.999934 | 0.000066 | 1 | 192 |
| [illegible] | 0.000 | 0.008 | 0.021 | 0.897 | 0.985 | 0.998 | 0.999907 | 0.000093 | 1 | 128 |

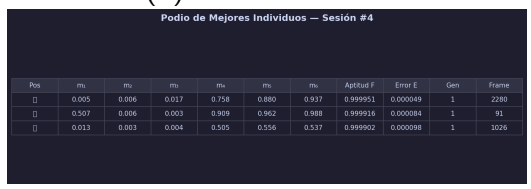
(b) Podio Sesión 2



Podio de Mejores Individuos — Sesión #3

| Pos | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | Aptitud F | Error E | Gen | Frame |
|---|---|---|---|---|---|---|---|---|---|---|
| [illegible] | 0.502 | 0.000 | 0.009 | 0.937 | 0.991 | 0.997 | 0.999951 | 0.000049 | 1 | 617 |
| [illegible] | 0.502 | 0.959 | 0.966 | 0.879 | 0.958 | 0.984 | 0.999905 | 0.000095 | 1 | 76 |
| [illegible] | 0.502 | 0.959 | 0.966 | 0.898 | 0.958 | 0.983 | 0.999880 | 0.000120 | 1 | 97 |

(c) Podio Sesión 3



Podio de Mejores Individuos — Sesión #4

| Pos | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | Aptitud F | Error E | Gen | Frame |
|---|---|---|---|---|---|---|---|---|---|---|
| [illegible] | 0.005 | 0.006 | 0.017 | 0.758 | 0.880 | 0.937 | 0.999951 | 0.000049 | 1 | 2280 |
| [illegible] | 0.507 | 0.006 | 0.003 | 0.909 | 0.962 | 0.988 | 0.999916 | 0.000084 | 1 | 91 |
| [illegible] | 0.013 | 0.003 | 0.004 | 0.505 | 0.556 | 0.537 | 0.999902 | 0.000098 | 1 | 1026 |

(d) Podio Sesión 4

Figura 1.8: Podio de resultados: mejores vectores de tensión $[m_1, \ldots, m_6]$ encontrados.

Los resultados expuestos confirman que el Algoritmo Genético es capaz de converger a soluciones de alta aptitud ($F > 0.98$) en menos de 20 generaciones, garantizando una mímica facial fluida y natural en el gemelo digital del robot EVA.

# Capítulo 2

# Conclusiones y recomendaciones

## 2.1. Conclusiones

La realización de este proyecto permitió validar el uso de Algoritmos Genéticos como una alternativa eficiente a los modelos matemáticos tradicionales en robótica blanda.

### 2.1.1. Aprendizajes

Se comprobó que los métodos heurísticos permiten gestionar la no linealidad de la silicona de forma efectiva en tiempo real.

### 2.1.2. Mejoras en Retrospectiva

De repetir el desarrollo, se implementaría una tasa de mutación adaptativa para mejorar la precisión.

### 2.1.3. Nuevos Proyectos

Los resultados obtenidos sientan las bases para el control físico del robot EVA y hibridación con IA.

## 2.2. Recomendaciones

Debido a la arquitectura basada en SQLite, el sistema es altamente escalable. Se recomienda migrar el gemelo digital a un modelo 3D realista usando Blender.

# Capítulo 3

# Anexos

## 3.1. Base de Conocimientos y Modelo del Robot

El catálogo de parámetros y reglas cinemáticas se almacena en el archivo `eva_conocimiento.db`.

## 3.2. Interfaz del Sistema en Ejecución

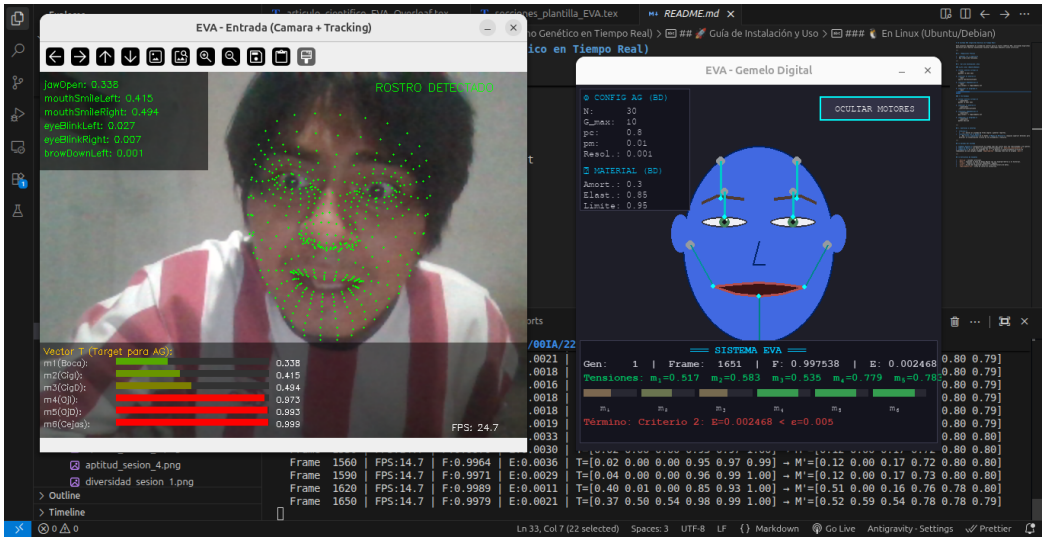



Figura 3.1: Captura del sistema EVA operando en tiempo real.

## 3.3. Repositorio de Código Fuente

`https://github.com/LuisAlbertoB/Algoritmo-genetico-para-operar-rostros-mecatronico git`

# Bibliografía

[1] SciTechDaily. Ameca: The world's most advanced humanoid robot practicing random facial expressions, 2024. Accesado el 2026.04.09. Disponible en https://scitechdaily.com/.